TRMD User Guide

English text edition. Embedded screenshots and image-only labels are omitted; follow the translated written steps.

A Telegram downloader with resumable downloads.

Tip:

Recommended proxy provider: Visit the site.

Bug reports with detailed reproduction steps are welcome.

Paid customizations, remote setup, training, and support are available.

Important:

The project is not obligated to resolve issues for you.

If you **cannot find a solution to your problem**:

Contact the developer for **paid** support.

We recommend watching the video tutorial and consulting this README and the FAQ and solutions.

Join the TRMD discussion group to search its chat history for similar issues. Repeated questions may not receive a response.

Features

Tip:

Bot required indicates that a bot must be configured.

Feature	Description
Restricted downloads	Download media from chats that restrict saving content.
Resumable transfers	Downloads and uploads can resume after interruption.
Web dashboard	Monitor download and upload progress in a browser.
Ready to use	Windows and Linux binaries are available, and the app can be deployed with Docker.
Batch downloads	Bot required Download entire channels and optionally their comments, with custom keyword, type, and date filters.
Guided setup	Bot required Create tasks through step-by-step prompts in the help menu.
Message forwarding	Bot required If a channel restricts saving content, forwarding falls back to download-and-upload.
File uploads	Bot required Batch-upload local files to Telegram.
Live monitoring	Bot required Monitor new channel messages and download or forward them automatically.

Disclaimer

This project is released as open-source software under the `GNU GPL-3.0` license.

Users are solely responsible for all use of this software and any resulting consequences. The developer accepts no liability for any use or its consequences.

This project is provided "as is," without warranties of any kind, express or implied.

Author	Link
Gentlesprite	GitHub
YouTube tutorial	Watch
Telegram group	Join
Support the developer	Support

1.0. Downloads

Platform	Download	Notes
GitHub	Releases	—
Gitee	Releases	Source code only
Lanzou Cloud	Download	Password: `ceze`
Telegram group	Join	Group files

1.1. Recommended terminal (optional)

Click to expand

1. _For Windows 11 users_, `Windows Terminal` is installed by default. You can **skip** the download step and go directly to step 3 (setting `Windows Terminal` as the **default terminal**).

2. _For Windows 10 users_, we recommend using `Windows Terminal` as the **default terminal**. This is optional and does not affect the software. `Windows Terminal` provides a better display and interaction experience and helps prevent **garbled text**.

Download `Windows Terminal` from one of the following sources:

Platform	Download
Microsoft Store	Download
GitHub	Releases

3. After installation, press `Win+R`, type `wt`, and press Enter. Set `Windows Terminal` as the **default terminal** before starting the software. See the screenshots below:

2.0. Quick start

2.1. Apply for a Telegram API ID and hash

Important:

Applying for a Telegram API ID and hash requires an IP address with a good reputation. If you cannot apply, you can use the `api` details provided in the description of the YouTube tutorial or Bilibili video; then you can skip the application steps below.

1. Go to **https://my.telegram.org/auth**.

2. Enter the **phone number associated with your Telegram account**. Include the country code first, for example, `+12223334455`, where `+1` is the country code. Click `Next`.

3. Open your Telegram client and find the message from Telegram. Enter its verification code in the `Confirmation code` field, then click `Sign in`.

4. Click `API development tools` and complete the form as prompted.

5. After applying, save your `api_hash` and `api_id`. **Never share them with anyone.**

2.2. Optional: apply for and use a Telegram bot `bot_token`

Tip:

A configured bot supports most use cases, such as downloading an entire channel or monitoring and forwarding channel messages. See the bot commands for your use case.

This also allows you to deploy `TRMD` on a server or NAS using `Windows`, `Linux`, or `Docker`.

Click to expand

2.2.1. Apply for a bot

1. Go to https://t.me/BotFather.

2. When prompted to open Telegram Desktop, click **Open Telegram Desktop**, as shown below:

If this dialog does not appear, install the **Telegram client** and try again.

3. Click **Start**, as shown below:

4. In the current **chat**, enter `/newbot` and press Enter, as shown below:

BotFather will reply: `"Alright, a new bot. How are we going to call it? Please choose a name for your bot."` Enter a display name for your bot, as shown below:

5. This is the display name, not the unique username. Choose any name; you can change it later with `/setname`.

6. Next, choose a **unique username** for the bot. It must end in `bot`; for example, `HelloWorld_bot` and `HelloWorldbot` are both valid. If the username is already taken, choose another one. BotFather will show `"Sorry, this username is already taken. Please try something different."` when a name is unavailable.

If you see the message shown above, the username is **taken** and you need to **choose another one**.

7. Continue until BotFather displays `"Done! Congratulations on your new bot. . ."`, as shown below:

This means the bot was created successfully. The red box indicated by the arrow contains your `bot_token`. **Never share it with anyone.**

2.2.2. Using the bot

1. After creating the bot, when the software asks `Enable bot (requires bot_token)? - y|n (default n)`, enter `y` if you want to use it, as shown below:

Enter the `bot_token` you obtained in step 7 of Apply for a bot in the field indicated by the arrow, then press Enter to complete setup.

2. After setup is complete and the software starts, wait for the message `Bot started successfully.` This indicates that the bot is ready to use, as shown below:

3. In the Telegram client, find your conversation with `BotFather` and locate the message from step 7 of Apply for a bot, or find your bot chat another way, as shown below:

Click the location indicated by the arrow to open your bot chat.

4. Click **Start**, as shown below:

You should receive a message from your **bot**, as shown below:

If you do not receive a message, try sending the bot any command.

5. The bot currently supports the following commands:

Command	Usage	Description
`/help`	Send `/help` to the bot.	Show the **available** commands.
`/download`	`/download link1 link2 link3 linkN` or `/download channel_link 1 100`	Create a **new** download task using either specific links or a message range.
`/table`	Send `/table` to the bot.	Print current download statistics in the **terminal**.
`/forward`	`/forward https://t.me/A https://t.me/B 1 100`	Forward messages from **channel A** to **channel B**, from message ID `1` through `100`.
`/exit`	Send `/exit` to the bot.	**Exit** the software.
`/listen_download`	`/listen_download https://t.me/A https://t.me/B https://t.me/N`	**Monitor** new messages in **channels A, B, and N** and download them.
`/listen_forward`	`/listen_forward https://t.me/A https://t.me/B`	**Monitor** new messages in **channel A** and forward them to **channel B**. If channel A restricts saving content, fall back to download-and-upload.
`/listen_info`	Send `/listen_info` to the bot.	Show the configured listeners.
`/upload`	`/upload local_file target_channel`	Upload a **local file** to a **target channel**.
`/upload_r`	`/upload_r local_folder target_channel`	Recursively upload a folder, including its subfolders, to a **target channel**.
`/download_chat`	`/download_chat channel_link`	Retrieve all channel content, filter it using custom criteria, and download the selected items.
`/auth`	`/auth`, `/auth user_id`, or `/auth @username`	Add, remove, or list users authorized to use the bot. Accepts a user `ID` or `@username`.

Other features:

• (Version ≥`v1.8.7`) Forward a message containing a video, photo, audio, voice message, GIF, document, video note, or live photo to the bot to create a download task.

• This feature ignores custom download-type settings so the forwarded media can be retrieved immediately.

• Forwarded messages are treated as new messages and receive independent filenames. Duplicate-file detection is not supported; manage repeated forwards of the same content carefully.

• This feature is intended for convenience. For messages that cannot be forwarded or downloaded this way, use the appropriate command for your needs.

2.2.3. Commands

/help

Menu

/download

Version notes:

Version	Description
≥v1.6.3	Resumable transfers are supported in all download scenarios, improving stability and reliability on poor networks.

• Method 1:

• image

• Once you send a valid download command, the terminal creates the corresponding download task, as shown below:

• image

• Method 2:

• Added in ≥v1.5.1, this method downloads a range from a **single link**. Specify the **channel link**, **starting ID**, and **ending ID**:

```
# Syntax:
/download channel_link start_id end_id
# Example:
/download https://t.me/test 1 500
# Downloads media from message ID 1 through ID 500 in https://t.me/test.
```

/table

Note: This table shows **live status**, not the **final download results**. It is updated each time you use it based on the **current** download records.

The **link statistics table** is shown below:

Note: Due to an early design limitation, the **link statistics table** has the following characteristics:

• The **link statistics table** counts **all supported types**; it does **not** limit the count to the types currently selected by the user.

• When counting **media in comments**, the table could report an incorrect total: the **total** could be 1, less than the number already downloaded, resulting in a **completion rate** above 100%. This issue was fixed in ≥v1.5.9.

• If the user has **not selected all supported types**, a completion rate below 100% after the selected types finish (or after requesting the **link statistics table** from the bot) means the link contains other types that were not selected. The selected types are already complete, so this is expected. Since ≥v2.0.2, download types marked as skipped in bot settings or the configuration file, as well as unsupported types, are excluded from the total.

• Since ≥v1.6.5, you can use this command to control at runtime **whether to export tables of selected types** when the app exits.

The **count statistics table** is shown below:

/forward

□□ **Note:**

Whether a message can be forwarded depends on whether the channel has **Restrict Saving Content** enabled.

If **forwarding is unavailable**, the **bot** provides a **Download** button and, since ≥`v1.6.7`, a **Download and upload** button in the **chat**.

□□ **Version notes:**

Version	Description
≥`v1.6.9`	`/forward` supports filtering by forwarding type. Change this under `[Help]` → `[Settings]` → `[Forwarding settings]`.
≥`v1.7.5`	To complete restricted forwarding, the download task created during **download-and-upload** ignores the download-type restrictions in the configuration file. In this case, `/forward` **cannot** apply the type filters from `[Forwarding settings]`.
≥`v1.8.5`	To complete restricted forwarding, the download task created during **download-and-upload** ignores the duplicate-link check.

• Forward messages with this syntax:

```
/forward channel_A channel_B start_id end_id
```

• Example:

• This example forwards messages 1 through 500 from the **channel** `https://t.me/test` to the **channel** `https://t.me/test2`.

```
/forward https://t.me/test https://t.me/test2 1 500
```

• Forward to **Saved Messages** (use either form):

• Method 1: use `me` or `self` to refer to Saved Messages:

```
/forward https://t.me/test me 1 500

/forward https://t.me/test self 1 500
```

• Method 2: use `https://t.me/username`:

• The username is the @ name shown in your account profile. You must set one yourself before using this method.

• For example, if your profile shows `@developer`, the link to your account is `https://t.me/developer`.

```
/forward https://t.me/test https://t.me/developer 1 500
```

• Both methods forward messages 1 through 500 from the **channel** `https://t.me/test` to **Saved Messages**.

• Forward Saved Messages to a channel:

• Method 1:

```
/forward me https://t.me/test 1 500
```

• Method 2:

```
/forward https://t.me/developer me 1 500
```

• Both methods forward messages 1 through 500 from **Saved Messages** to the **channel** `https://t.me/test`.

/exit

/listen_download

• `/listen_download` listens for new messages at the specified link and downloads them as they arrive.

• The bot confirms when a valid listening command has been received.

• When downloadable content appears in the monitored channel, the bot automatically starts a download.

Register a download listener

```
/listen_download https://t.me/A
```

Remove a download listener

Removal method

Send the same command used to create the listener again. The bot displays an inline keyboard for removing it; click Confirm to proceed.

Since ≥`v2.0.2`, you can also remove a download listener with the button in the web dashboard's `Listeners` section.

```
/listen_download https://t.me/A
```

Register multiple download listeners

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

Remove multiple download listeners

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

/listen_forward

□□ **Version notes:**

Version	Description
≥`v1.6.7`	When restricted forwarding is detected, the bot automatically uses **download-and-upload** (enabled by default). After the upload completes, you can choose whether to **delete the local file** (disabled by default). Change these options under `[Help]` → `[Settings]` → `[Upload settings]`.
≥`v1.6.9`	`/listen_forward` supports filtering by forwarding type. Change this under `[Help]` → `[Settings]` → `[Forwarding settings]`.
≥`v1.7.5`	To complete restricted forwarding, the download task created during **download-and-upload** ignores the download-type restrictions in the configuration file.
≥`v1.8.5`	To complete restricted forwarding, the download task created during **download-and-upload** ignores the duplicate-link check.

• `/listen_forward` listens for new messages at the specified link and forwards them as they arrive.

• As with `/forward`, whether a message can be forwarded directly depends on whether the channel has **Restrict Saving Content** enabled.

• Since ≥`v1.6.7`, restricted messages can also be forwarded by downloading and uploading them.

• The bot confirms when a valid listening command has been received.

• Whenever **any** new content appears in the monitored channel, it is automatically forwarded to the specified destination.

Listener behavior

• **One listener per channel:**

• A channel can have only one active listener mode (**download** or **forward**) at a time.

• For `/listen_forward`, the restriction applies to the **source channel** being monitored.

• The destination channel is **not** restricted and can still have a `/listen_download` task.

• **Operational limits:**

• If `channel_A` is monitored for forwarding to `channel_B`:

• A download listener can be set up on `channel_B` **at the same time**.

• However, the download listener on `channel_B` will not process messages forwarded there by the listener on `channel_A`.

• **Switching listener modes:**

• First cancel the existing listener using the **same command** used to create it.

• Then create the **new listener**.

Register a forwarding listener

```
/listen_forward https://t.me/A https://t.me/B
```

Remove a forwarding listener

Removal method

Send the same command used to create the listener again. The bot displays an inline keyboard for removing it; click Confirm to proceed.

Since ≥`v2.0.2`, you can also remove a forwarding listener with the button in the web dashboard's `Listeners` section.

```
/listen_forward https://t.me/A https://t.me/B
```

/listen_info

• Use `/listen_info` to view the **current listener information**. Send the command without arguments:

```
/listen_info
```

/upload

□□ **Version notes:**

Version	Description
≥v1.7.9	Resumable transfers are supported in all upload scenarios, improving stability and reliability on poor networks.
≥v1.8.4	The /upload command accepts me or self as the destination to refer to **Saved Messages**.

• Use /upload to upload local files to a specified channel.

• Folder uploads are supported when the specified path is a folder (≥v1.7.1).

• _When the path is a **folder**, **all** files in that folder are uploaded._

• Upload syntax:

```
/upload local_file_or_folder destination_channel
```

Notes

Telegram sets a **maximum size** for each uploaded file:

• **Regular accounts**: up to 2000 MiB (about 2 GB) per file.

• **Telegram Premium accounts**: up to 4000 MiB (about 4 GB) per file.

Windows

Upload a file

```
/upload C:\files\video.mp4 https://t.me/test
```

Upload a folder

Requires version ≥v1.7.1.

```
/upload C:\files https://t.me/test
```

Linux

Upload a file

```
/upload /home/username/files/video.mp4 https://t.me/test
```

Upload a folder

Requires version ≥v1.7.1.

```
/upload /home/username/files https://t.me/test
```

/upload_r

□□ **Version notes:**

Version	Description
≥v1.8.4	The /upload_r command accepts me or self as the destination to refer to **Saved Messages**.

• /upload_r extends /upload for **batch uploads** and recursively processes directory structures. Unlike /upload:

• It accepts a **folder path** as its second argument and automatically **searches that directory** and **all its subdirectories** for files.

```
/uploadr C:\files https://t.me/test
```

• If the **second argument** is a **file path**, the command switches to `/upload`'s single-file upload mode.

```
/uploadr C:\files\video.mp4 https://t.me/test

/upload C:\files\video.mp4 https://t.me/test
```

These two commands are functionally equivalent. The app detects the file type and uses the appropriate upload method.

/download_chat

• Use `/download_chat` to download a specified channel.

• Unlike `/download`, `/download_chat` supports custom content filters through the inline keyboard sent by the bot.

• This command can download chats whose specific message links cannot be copied.

• After sending the command, use the inline keyboard in the bot's reply to configure filters, start the task, or cancel it.

• You cannot create another `/download_chat` task until the previous one has been started or canceled.

• Since ≥`v1.7.5`, tasks created with `/download_chat` follow the filters set in the inline keyboard and ignore all configuration-file rules, including configured download-type filters.

• Supported filters:

Filter	Default
Date range	`date of the first message–date of the last message`
Media type	`video`, `photo`, `document`, `audio`, `voice`, `animation (GIF)`, `video note`, `live photo`
Keyword matching	–
Include comments	`off`

• Syntax for downloading a channel:

```
/download_chat channel_link
```

• After sending the command, you can optionally configure filters to suit your needs.

• After configuring the filters, you must **manually click** `Start task` or `Cancel task` to continue or stop the process. Otherwise, the command will **remain pending and prevent another** `/download_chat` **command from running.**

• Download media from a conversation between a user and a bot:

A specific message link cannot be copied from a conversation between a user and a bot. Use this method when such a conversation prevents forwarding or downloading.

• First, find the bot's username by clicking its name in the chat. This example uses `@developer_bot`.

• Form the bot link as `https://t.me/developer_bot`.

```
/download_chat https://t.me/developer_bot
```

• Download **Saved Messages** (use either form):

• Method 1 (use either command):

```
/download_chat me

/download_chat self
```

• Method 2:

• First, find your username. This example assumes your profile has the username `@developer`.

• Form the Saved Messages link as `https://t.me/developer`.

```
/download_chat https://t.me/developer
```

• Since ≥`v1.8.8`, `/download_chat` supports filtering by keyword:

• The command checks message `text` and `caption` and downloads matching messages that contain the configured keywords.

• After sending the command, click [■ `Set matching keywords`] to put the bot into keyword-entry mode.

• You can enter one or more keywords. Separate multiple keywords with spaces, for example:

```
my favorites
```

```
study materials health nutrition
```

• If you send an already-added keyword again, the bot displays an [■■ `inline keyboard`] with options to [■■ `remove`] or [■■■■■ `ignore`] that keyword.

• If you send the same existing keyword multiple times in a row, only the first occurrence triggers the remove option.

• On the [■ `Set matching keywords`] screen:

• Click [■ `Confirm keywords`] to add all entered keywords.

• Click [■ `Cancel`] to clear all entered keywords.

• Click [■ `Back`] to return to the main screen without making changes.

• Comment settings:

• **Include comments** toggle:

• Off by default.

• When enabled, the button reads [■ `Include comments`]. If a matching message has a comments section, the comments are also checked.

• When disabled, the button reads [■ `Include comments`], and comments are ignored.

• **When to use comments:**

• Some channels have a comments section beneath their messages. This optional feature lets you decide whether to retrieve those comments.

• **Comment matching and filtering:**

• When comments are enabled, comments are retrieved whenever a message has a comments section, even if the message itself was excluded by the `media type` filter (≥`v2.0.8`).

• **Comments are filtered only by the** `media type` **selected for the current task. Other filters, such as the date range, are ignored.**

• **Processing time:**

• With comments enabled, the app also reads comments for matching messages. Large message counts may increase task duration, so please wait for processing to finish.

/auth

□□ **Permissions:**

The `/auth` command is **for** `root` **only** (`root` is the logged-in account). Other users who send it will be told that the command is restricted to `root`.

• Use `/auth` to manage **users authorized to use the bot**. You can add users by `user ID` or `@username`, view the authorized-user list, and remove users.

• View the current list of authorized users by sending the command without arguments:

```
/auth
```

• Add an authorized user by specifying a `user ID` or `@username` (`root` only):

```
/auth 1234567890

/auth @username
```

• Remove an authorized user: open the bot menu and select `[■ Authorized users]` → `[■ Add/remove authorized users]`. In the authorized-user list, click the corresponding `[■■ Remove authorization]` inline button.

2.3. Configuration file reference

User configuration file

[!NOTE]

You usually do not need to configure this file manually. This section explains the meaning of each parameter.

On startup, the app creates `config.yaml` in its directory and guides you through configuring the settings.

If you **edit** `config.yaml` manually, read the parameter descriptions below carefully.

Use standard (half-width) quotation marks and colons in the file, and include a space after each colon.

```
api_hash: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
api_id: 'xxxxxxxx'
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type:
  - video
  - photo
  - document
  - audio
  - voice
  - animation
  - video_note
  - live_photo
is_shutdown: true
links: D:\path\where\your\link\files\save\content.txt
max_retries:
  download: 5
  upload: 3
max_tasks:
  download: 3
  upload: 3
proxy:
  enable_proxy: true
  hostname: 127.0.0.1
  scheme: socks5
  port: 10808
  username: null
  password: null
save_directory: F:\directory\media\where\you\save
session_directory: F:\directory\session\where\you\save
temp_directory: F:\directory\temp\where\you\save
```

Parameter reference

Parameter	Description
`api_hash`	Your Telegram API `api_hash` credential.
`api_id`	Your Telegram API `api_id` credential (enclose it in quotes).
`bot_token`	Optional bot token. Bot features are unavailable without it. Get one for free from BotFather.
`download_type`	Media types to download. See the table of supported media types for this parameter.
`is_shutdown`	Whether to shut down automatically after downloads finish. Supported values: `true` or `false`.
`links`	Path to a text file containing links, one per line. Do not quote the path; create the file before starting the app.
`max_retries`	Maximum retries for tasks, with separate values for `download` and `upload`.
`max_tasks`	Maximum concurrent tasks, with separate limits for `download` and `upload`.
`proxy`	Proxy settings. See the `proxy` parameter table for its fields. To disable the proxy, set `enable_proxy` to `false`. Keep a space after the colon or YAML parsing will fail.
`save_directory`	Directory for downloaded media (wildcards are supported).
`session_directory`	Directory for session files (wildcards are supported).
`temp_directory`	Directory for cached files (wildcards are supported).

Media types supported by `download_type`

Media type	Description
`video`	Video.
`photo`	Photo.
`document`	Document.
`audio`	Audio.
`voice`	Voice message.
`animation`	Animation (GIF).
`video_note`	Video note.
`live_photo`	Live Photo (treated as `photo` when this type is not specified and `photo` is enabled).

`proxy` parameter fields

Setting	Description
`enable_proxy`	Whether to enable the proxy. Supported values: `true` or `false`.
`scheme`	Proxy protocol, such as `socks5` or `http`.
`hostname`	Proxy server `IP` address or domain name.
`port`	Proxy server port.
`username`	Optional proxy authentication username.
`password`	Optional proxy authentication password.

Wildcards

Since ≥`v1.7.4`, the `save_directory` parameter supports wildcards that dynamically generate storage paths. During downloads, the app replaces each wildcard with its corresponding value, allowing files to be organized automatically.

• The wildcards currently supported by `save_directory` are:

Wildcard	Meaning
`%CHAT_ID%`	Inserts the actual `channel ID` into the path.
`%CHAT_NAME%`	Inserts the actual `channel name` into the path (≥`v1.9.6`).
`%MIME_TYPE%`	Inserts the actual `file type` into the path.
`%MESSAGE_ID%`	Inserts the actual `message ID` into the path (≥`v2.1.1`). For a `media group`, it uses the lowest-to-highest member ID range, such as `123-129`.
`%MESSAGE_ID_WITH_ENTITIES%`	Uses the same path substitution as `%MESSAGE_ID%` and also saves the message text and formatting entities as a `json` file (`{range_or_ID}_entities.json`) (≥`v2.1.1`).

• Example 1:

```
F:\directory\media\%CHAT_ID%
```

• Example 2 (≥`v1.9.6`):

```
F:\directory\media\%CHAT_NAME%
```

• Example 3:

```
F:\directory\media\%MIME_TYPE%
```

• Example 4:

```
F:\directory\media\%CHAT_NAME%\%CHAT_ID%\%MIME_TYPE%
```

Global configuration file

[!NOTE]

You usually do not need to edit the global configuration file manually. This section explains its parameters.

Some parameters can be changed through the bot settings.

```
authorized_users: []
console_log_level: WARNING
export_table:
  count: false
  link: false
file_log_level: INFO
forward_type:
  animation: true
  audio: true
  document: true
  photo: true
  text: true
  video: true
  voice: true
  video_note: true
  live_photo: true
notice: false
upload:
  delete: false
  download_upload: true
```

Parameter reference

Parameter	Description
`authorized_users`	Users authorized to use the bot with the `/auth` command.
`console_log_level`	Minimum log level shown in the terminal, such as `WARNING`.

Parameter	Description
`export_table`	Whether to export statistics tables when the app exits: `count` for download counts, `link` for download links, and `upload` for upload statistics.
`file_log_level`	Minimum log level recorded in `TRMD_LOG.log`, such as `INFO`.
`forward_type`	File types that `/listen_forward` and `/forward` can forward. See the list of media types supported by `forward_type`.
`notice`	Whether the bot sends a startup notification (welcome message).
`upload`	How to handle protected channel content during forwarding: `delete` controls whether the local file is deleted after download and upload (off by default); `download_upload` controls whether restricted forwarding falls back to download-and-upload (on by default).

Media types supported by `forward_type`

Media type	Description
`text`	Text.
`video`	Video.
`photo`	Photo.
`document`	Document.
`audio`	Audio.
`voice`	Voice message.
`animation`	Animation (GIF).
`video_note`	Video note.
`live_photo`	Live Photo (treated as `photo` when this type is not specified and `photo` is enabled).

Global configuration file location

Windows

```
%APPDATA%/TRMD/.CONFIG.yaml
```

Linux

```
~/.config/TRMD/.CONFIG.yaml
```

2.4. Notes

Click to expand

How to copy a link

Right-click the media you want to save and select **Copy message link**, as shown below:

Format of the `links` text file

As shown below, put one link on each line:

Common `links` file formatting mistakes

See the example below:

Downloading links from private channels

• The Telegram account currently logged in must be a member of the private channel; otherwise, downloads will fail.

• This restriction is imposed by Telegram's servers and cannot be bypassed by any downloader.

Duplicate links

Description

• As shown above, a submitted download task may contain multiple links with the **same prefix but different parameters**, such as `?comment`, `?single`, or `?single&comment`.

Cause

• When a link includes the `?comment` parameter, the app downloads the **original message and its comments**.

• Submitting another link with the **same prefix but without** `?comment` can add the **same resource to the download queue more than once**.

• If the earlier task has not finished, submitting the same resource again can cause a **task conflict** and a download error.

Solution

• **Since** `v2.0.8`**, the app automatically handles resource conflicts caused by duplicate links.**

• **Submit just one complete link** (for example, one with `?comment`). The app handles both the **original content and its comments**, so there is no need to submit another version without parameters.

• **Avoid submitting the same resource more than once.** Make sure the `t.me/c//` portion is unique for each link to prevent redundant tasks.

• Links with the same **channel name** and **message ID** point to the **same resource**, even if their query parameters differ.

• Note the different purposes of these parameters:

• `?comment` indicates whether to download **comments**.

• `?single` selects one item from a **media group**.

• `?single&comment` selects one item with the specified message ID from the comments.

Telegram link fields

Field	Meaning
`?comment`	A link to **comments**.
`?single`	A **single-item** link.
`?single&comment`	A **single-item** link in **comments**.
`/c`	A **private channel** link.
`https://t.me/TEST/111/666`	Link to **topic** `111` in channel `TEST`.
`https://t.me/c/1111111111/333/666`	Link to **topic** `333` in private channel `1111111111`.

Link details

Telegram link structure

Channel type	Link structure
Public channel	`https://t.me/channel_name/message_id`
Private channel	`https://t.me/c/channel_id(10 digits)/message_id`
Topic channel	`https://t.me/channel_name/topic_id/message_id`
Private topic channel	`https://t.me/c/channel_id(10 digits)/topic_id/message_id`

Telegram link formats

"All" means that if files were sent together as a group, one link downloads **every file in that group**.

"Media" means the media types selected with the `download_type` setting in `config.yaml`.

Link	Resolved channel	Message ID	Description
`https://t.me/TEST/111`	`TEST`	`111`	Download **all media** from this link.
`https://t.me/TEST/111?single`	`TEST`	`111`	Download **one media item** from this link.
`https://t.me/TEST/111?comment=666`	`TEST`	`111`	Download **all media** from this link and **all media in its comments** (if the link has no comments, it is handled as `https://t.me/TEST/111`).
`https://t.me/TEST/111?single&comment=666`	`TEST`	`111`	Download **one media item** from the linked **comments**.
`https://t.me/c/1111111111/666`	`-1001111111111`	`666`	Download **all media** from this **private channel** link.
`https://t.me/TEST/111/666`	`TEST`	`666`	Download **all media** from this **topic** link.
`https://t.me/c/1111111111/333/666`	`-1001111111111`	`666`	Download all media from this **private topic** link.

Downloading files from comments

Copy a link with the `?comment` parameter

Usually, you can copy the link to any message in the comments.

How comment links are handled

Standard links (without `?comment`)

• Downloads only the **message content** (the original post in the channel or group).

• **Does not include comments**, even if the original message has comments.

Links with the `?comment` parameter

• Downloads the **message content and all associated comments** (the complete thread).

• If the original message has **no comments** (for example, it is a channel message or comments are disabled), only the **message content** is downloaded, as with a link without parameters.

Recommended link formats when not downloading comments

Use the formats shown below:

Channel type	Link
Public channel	https://t.me/xxx/111
Private channel	https://t.me/c/xxxxxxxxxx/111
Topic channel	https://t.me/xxx/xxx/111
Private topic channel	https://t.me/c/xxxxxxxxxx/xxx/111

How `?single` and `?single&comment` affect downloads

Overview

Since `v1.5.8`, links containing `?single` or `?single&comment` enable **single-item download mode**, designed for these use cases:

- Prevents **duplicate downloads** when `/listen_download` detects grouped files (`v1.5.8` and later).
- Extracts **one specific file** from a grouped media message.
- Downloads **one specific media item** from comments, which can be useful when there are many comments.

Parameter behavior

Parameter format	Download scope	Use case
`xx?single`	Download only the **media item corresponding to** `xx` in the message	Extract one item from a grouped photo/video message
`?single&comment=xx`	Download only the **media item corresponding to** `xx` in the comments	Retrieve one photo/video shared in the comments

Version compatibility

- This feature is supported **only in** `v1.5.8` **and later**.
- Older versions may ignore these parameters and download all content.

Best practices

Downloading a single file

When a message contains multiple media files, append `?single` to its standard link to retrieve just one file:

```
https://t.me/c/123456789/123?single
```

Downloading one file from comments

To download one file from comments, use the combined parameter format:

```
https://t.me/c/123456789/123?single&comment=xx
```

Parameter compatibility

• Avoid submitting both full-content and single-item links for the same message. Duplicate-link conflicts are handled automatically in `v2.0.8` and later.

• Single-item mode cannot be combined with comment-download mode (`?comment`).

2.5. Software update tutorial

3.0. Running in production

**Python 3.13.2 is recommended** for this project to avoid runtime errors that may occur with other Python versions.

Windows

Install `python` and `git` and configure the **PATH environment variable**.

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

Linux

Clone the project and **change to its directory**.

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
```

**Upgrade** `pip` (recommended).

```
python3 -m pip install --upgrade pip
```

Create and activate a **virtual environment** (optional).

```
python3 -m venv venv
source venv/bin/activate
```

Install the **required dependencies** and wait for the installation to finish.

```
pip3 install -r requirements.txt
```

Run the app. Installation is complete; the following section contains additional notes.

```
python3 main.py
```

Notes

If you use a virtual environment, activate it before running the app each time.

From the project directory, **activate the virtual environment** before running the app.

```
source venv/bin/activate
python3 main.py
```

If you are told that **pip is not installed**, install it with:

```
sudo apt update
sudo apt-get install python3-pip
```

Updating

Open a terminal in the **project directory** and run this command to get the **latest version**:

```
git pull
```

A new version may use **new dependencies**, so update them after running `git pull`. If you use a **virtual environment**, activate it first.

```
pip3 install -r requirements.txt
```

4.0. Command-line options (advanced)

[!NOTE]

Since `v1.8.3`:

You can use **command-line options** to further customize the app's **runtime configuration** before starting it.

Click to expand

To set command-line options, open a terminal in the **application directory**, or open a terminal elsewhere and **change to the application directory**. If you use the app frequently, consider configuring your **PATH environment variable**.

The currently supported command-line options are listed below:

Short option	Long option	Description
`-h`	`--help`	Show help
`-v`	`--version`	Show version information
`-q`	`--quiet`	Skip the confirmation prompt for reconfiguring the file
`-c`	`--config`	Set the user configuration file path
`-s`	`--session`	Set the session file path
`-t`	`--temp`	Set the temporary-file path
`-p`	`--port`	Set the web dashboard port

Long and short options have the same effect.

4.1. Options

-h, --help

This option displays help.

• For production deployments (**complete the prerequisite steps first** in _“3.0. Running in production”_):

```
python3 main.py -h

python3 main.py --help
```

• For Windows users:

```
TRMD.exe -h
```

```
TRMD.exe --help
```

• For Linux users:

```
./TRMD -h

./TRMD --help
```

-v, --version

This option displays version information.

• For production deployments (**complete the prerequisite steps first** in _“3.0. Running in production”_):

```
TRMD.exe -v

TRMD.exe --version
```

• For Windows users:

```
TRMD.exe -v

TRMD.exe --version
```

• For Linux users:

```
./TRMD -v

./TRMD --version
```

-q, --quiet

Usage notes
1. This option skips the confirmation prompt for reconfiguring the file.
2. This option has no effect if the user configuration file is missing required parameters.
3. This option is temporary and is not saved.

• For production deployments (**complete the prerequisite steps first** in _“3.0. Running in production”_):

```
python3 main.py -q

python3 main.py --quiet
```

• For Windows users:

```
TRMD.exe -q

TRMD.exe --quiet
```

• For Linux users:

```
./TRMD -q

./TRMD --quiet
```

-c, --config

Usage notes
1. This option sets the path to the user configuration file.

Usage notes
2. **It lets multiple instances use separate configuration files without requiring a separate copy of the application.**
3. Specify a file that follows the format described in "User configuration file" and has a `.yaml` extension.
4. If the specified **file path is invalid**, the application uses its **default** settings.
5. This option is temporary and is not saved.

• For production deployments (**complete the prerequisite steps first** in _"3.0. Running in production"_):

The example below is for `Linux` (the same applies to `Windows`) and assumes the user configuration file is located at `/home/username/files/example.yaml`.

```
python3 main.py -c /home/username/files/example.yaml

python3 main.py --config /home/username/files/example.yaml
```

• For Windows users:

This example assumes the user configuration file is located at `C:\files\example.yaml`.

```
TRMD.exe -c C:\files\example.yaml

TRMD.exe --config C:\files\example.yaml
```

• For Linux users:

This example assumes the user configuration file is located at `/home/username/files/example.yaml`.

```
./TRMD -c /home/username/files/example.yaml

./TRMD --config /home/username/files/example.yaml
```

-s, --session

□□ Version notes:

Version	Description
≥`v1.8.5`	After you set `-s` or `--session`, its value is saved as `session_directory` in the user configuration file. You do not need to specify it again unless you want to change it.
≥`v2.1.1`	A `TRMDStorage.db` database is created in the session directory specified with `-s` or `--session`. It stores download and upload history for the web dashboard. Deleting or moving it affects only dashboard history, not account login.

Usage notes
1. This option sets the path to the session files.
2. After a successful login, the application creates a `sessions` folder in its working directory to save the current account information for faster future logins.
3. **This option avoids having to rename session files manually** when using multiple accounts. Specify a different path to select the account to log in with.
4. Specify a **folder** that may contain existing `.session` files. If the folder is empty or does not exist, a session file is **created there automatically** after login.
5. The setting is saved, so you do not need to specify it again (≥`v1.8.5`).
6. In addition to `.session` files, the session folder contains `TRMDStorage.db` (≥`v2.1.1`), which stores web dashboard download and upload history. Deleting or moving this database clears only the dashboard history; it does not affect account login.

• For production deployments (**complete the prerequisite steps first** in _"3.0. Running in production"_):

The example below is for `Linux` (the same applies to `Windows`) and assumes the session folder is located at `/home/username/files/session`.

```
python3 main.py -s /home/username/files/session

python3 main.py --session /home/username/session
```

• For Windows users:

This example assumes the session folder is located at `C:\files\session`.

```
TRMD.exe -s C:\files\session

TRMD.exe --session C:\files\session
```

• For Linux users:

This example assumes the session folder is located at `/home/username/files/session`.

```
./TRMD -s /home/username/files/session

./TRMD --session /home/username/files/session
```

-t, --temp

□□ **Version notes:**

Version	Description
≥`v1.8.5`	After you set `-t` or `--temp`, its value is saved as `temp_directory` in the user configuration file. You do not need to specify it again unless you want to change it.

Usage notes
1. This option sets the path for temporary files.
2. Temporary storage holds information about **downloads and uploads** while the application is running.
3. **Resumable transfers** rely on this cache. You can **choose a custom cache location to suit your needs**.
4. Specify a **folder**. If it is empty or does not exist, cache files are **created there automatically** at runtime.
5. The setting is saved, so you do not need to specify it again (≥`v1.8.5`).

• For production deployments (**complete the prerequisite steps first** in _"3.0. Running in production"_):

The example below is for `Linux` (the same applies to `Windows`) and assumes the temporary-file folder is located at `/home/username/files/temp`.

```
python3 main.py -t /home/username/files/temp

python3 main.py --temp /home/username/temp
```

• For Windows users:

This example assumes the temporary-file folder is located at `C:\files\temp`.

```
TRMD.exe -t C:\files\temp

TRMD.exe --temp C:\files\temp
```

• For Linux users:

This example assumes the temporary-file folder is located at `/home/username/files/temp`.

```
./TRMD -t /home/username/files/temp

./TRMD --temp /home/username/files/temp
```

-p, --port

Usage notes
1. The web dashboard is enabled by default and lets you view download progress in a browser.
2. The dashboard's web service runs in the background, so you can continue using the terminal.
3. `Web configuration` details are shown in the terminal; use them to enter the username and password when opening the dashboard.
4. **Closing the browser window does not stop the application**. The dashboard closes when the application exits.
5. This option sets the dashboard **port** (`0`–`65535`). The default is `2921`; if the port is occupied, an available port is assigned automatically.
6. This option is temporary and is not saved.

• For production deployments (**complete the prerequisite steps first** in _"3.0. Running in production"_):

This example uses port `1024`.

```
python3 main.py -p 1024

python3 main.py --port 1024
```

• For Windows users:

This example uses port `1024`.

```
TRMD.exe -p 1024

TRMD.exe --port 1024
```

• For Linux users:

This example uses port `1024`.

```
./TRMD -p 1024

./TRMD --port 1024
```

5.0. Running a compiled build

We **recommend** using `Python==3.13.2` in a **virtual environment** and **avoiding** unnecessary packages to **reduce** the size of the build output.

`Windows` build notes:

• **The Windows build enables `--clang`.**

• Install `Visual Studio` and select the `C++ Clang Compiler for Windows` component to provide `clang-cl`.

• Check the `Visual Studio` version: its `LLVM`/`clang` version must be **≥19**, because the `#embed` (C23) constant embedding used by Nuitka is supported only in that version and later.

• On `Windows`, run `build.py` from the **x64 Native Tools Command Prompt for VS**. Nuitka relies on the `MSVC` and `Windows SDK` build environment initialized by that prompt.

• Running from CMD or PowerShell causes a compile-time error because the required environment variables are missing. Nuitka no longer falls back to gcc/zig (fallback occurs only if MSVC cannot be detected at all).

Follow the same **prerequisites** as in _“3.0. Running in production”_.

Run the build command:

```
python build.py
```

6.0. Running with Docker

**Example configuration for Docker** (you must start Docker using the commands provided below):

```
api_hash: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
api_id: 'xxxxxxxx'
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type:
- video
- photo
- document
- audio
- voice
- animation
- video_note
- live_photo
is_shutdown: false
links: /app/TRMD/links.txt
max_retries:
  download: 5
  upload: 3
max_tasks:
  download: 3
  upload: 3
proxy:
  enable_proxy: true
  hostname: 192.168.1.10
  scheme: socks5
  port: 10808
  username: null
  password: null
save_directory: /app/downloads/%CHAT_NAME%/%MIME_TYPE%
session_directory: /app/sessions
temp_directory: /app/temp
```

The Docker container's working directory is /app, so configuration paths must be **paths inside the container**. The -v volume mounts in docker run map the corresponding files to host directories:

Setting	Container path (use in configuration)	Corresponding host path
save_directory	/app/downloads/%CHAT_NAME%/%MIME_TYPE%	./downloads/%CHAT_NAME%/%MIME_TYPE%
session_directory	/app/sessions	./sessions
temp_directory	/app/temp	./temp
links	/app/TRMD/links.txt	./config/links.txt

• _*Also mount ./form:/app/form for the application's internal statistics and form-export directory (/app/form)._

• _*The proxy hostname (192.168.1.10 in this example) is only a placeholder. Replace it with your proxy server's IP address (use ip addr or ifconfig to check your local network IP). If the environment can access Telegram directly, you can disable the proxy by setting enable_proxy to false._

Method 1:

• _Make sure docker is installed and available on your **PATH**._

• Use docker to pull the image from the remote repository. The TRMD version must be ≥2.0.2:

```
docker pull gentlesprite/telegram_restricted_media_downloader:latest
```

• On first use, run the container interactively to configure the application:

```
docker run -it --rm \
  --name trmd \
  -v ./config:/app/TRMD \
  -v ./sessions:/app/sessions \
  -v ./downloads:/app/downloads \
  -v ./temp:/app/temp \
  -v ./form:/app/form \
  -p 2921:2921 \
  -e TZ=Asia/Shanghai \
  gentlesprite/telegram_restricted_media_downloader:latest \
  python main.py --config /app/TRMD/config.yaml --port 2921
```

• If you are running `docker` through `wsl2` on `Windows`:

```
docker run -it --rm --name trmd -v ./config:/app/TRMD -v ./sessions:/app/sessions -v
  ./downloads:/app/downloads -v ./temp:/app/temp -v ./form:/app/form -e TZ=Asia/Shanghai
  gentlesprite/telegram_restricted_media_downloader:latest python main.py --config
  /app/TRMD/config.yaml --port 2921
```

• After configuration is complete, create and start the container:

```
docker run -it -d \
  --name trmd \
  -v ./config:/app/TRMD \
  -v ./sessions:/app/sessions \
  -v ./downloads:/app/downloads \
  -v ./temp:/app/temp \
  -v ./form:/app/form \
  -p 2921:2921 \
  -e TZ=Asia/Shanghai \
  gentlesprite/telegram_restricted_media_downloader:latest \
  python main.py --config /app/TRMD/config.yaml --port 2921
```

• If you are running `docker` through `wsl2` on `Windows`:

```
docker run -it -d --name trmd -v ./config:/app/TRMD -v ./sessions:/app/sessions -v
  ./downloads:/app/downloads -v ./temp:/app/temp -v ./form:/app/form -p 2921:2921 -e
  TZ=Asia/Shanghai gentlesprite/telegram_restricted_media_downloader:latest python main.py
  --config /app/TRMD/config.yaml --port 2921
```

Method 2:

• _Make sure `git`, `docker`, and `docker-compose` are installed and available on your **PATH**._

• Clone the repository with `git`:

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
```

• Change to the project directory:

```
cd Telegram_Restricted_Media_Downloader
```

• On first use, run the container interactively to configure the application:

```
docker-compose run --rm --service-ports --name trmd trmd
```

• After configuration is complete, create and start the container:

```
docker-compose up -d
```

After starting with either method, view the `Web configuration` to access the dashboard. Run `/help` in the terminal or send it to the bot in `Telegram`. It includes the port, username, password, and URL, as shown below:

Property	Value
Port	`2921`
Username	`cLJqKG3b`
Password	`AiJaKSObcRCZ`
URL	`http://127.0.0.1:2921`

Important:

In a `Docker` environment, the `IP` in the URL is the **container IP**, not the **host IP**. Access the dashboard using the host IP and port. `http://127.0.0.1:2921` is shown only as an example.

The username and password are generated randomly. Open http://127.0.0.1:2921 in a browser and enter them to sign in.

• **If a bot is configured**, you can also send `/help` to it in `Telegram` to view the `Web configuration` described above.

• The dashboard uses port `2921` by default. Use `-p` or `--port` to specify another port; if it is occupied, an available port is assigned automatically.

• The dashboard's web service runs in the background, so you can continue using the terminal. **Closing the browser window does not stop the application**; the dashboard closes when the application exits.

• After the first successful username-and-password login, the dashboard stores a `Cookie` in **this browser** that is valid for `30` days. This browser can access the dashboard without signing in again, even after the application restarts.

• For remote access, replace the `ip` address with your public `ip` and open port `2921` or your `custom` port.

Other commands:

• To view the application logs:

```
docker logs trmd
```

• When you are **finished using the application**, stop and remove the container:

```
docker stop trmd && docker rm trmd
```

7.0. Contact

Telegram @Gentlesprite

Email: Gentlesprites@outlook.com

8.0. Support the developer